

Station ALMA-7: Rescue Protocol

Functions · Closures · Optionals

03:14 station time. A meteorite has damaged the main computer of orbital station ALMA-7. The sensors are sending garbage, one module is running out of oxygen, and nobody knows where one of the crew members is. The onboard AI is offline. All that's left is a Swift Playground — and you.

Your mission: recover the data, stabilize the systems, and obtain the **launch code for the escape pod**. The code is built from the results of your functions. If even one function is wrong, the pod won't launch.

Topics: functions, argument labels, default parameter values, variadic parameters, inout, function types, closures, optionals (`if let` , `guard let` , `??` , optional chaining).

Submission: your completed `ALMA7_Starter.swift` (or a Playground with the same contents), with answers to the defense questions in the comment block at the end.

Station Rules

1. **The `!` operator is forbidden.** Every force unwrap in your code costs −0.5 points. Exception: the "Saboteur's Logbook" task, where `!` already appears in someone else's code.
2. You may not use `map` , `filter` , `reduce` , or `compactMap` — we haven't covered them yet. You'll write what they do yourself (and understand how they work).
3. Name your functions **exactly** as specified. Your code will be checked automatically.
4. Test every function with at least two calls and `print` .

Starter File

Download `ALMA7_Starter.swift` and paste its full contents into a new Xcode Playground (or run it with `swift ALMA7_Starter.swift`). **Do not modify the starter section.** Do not copy code from this PDF — PDFs break indentation and quotes.

The starter file gives you:

Item	What it is
<code>typealias Reading</code>	<code>(sensor: String, value: Int)</code>
<code>splitOnce(_:by:)</code>	Splits a string at the first separator, returns <code>(String, String)?</code>
<code>rawLog</code>	16 raw sensor lines like <code>"02:87"</code> , <code>"TEMP:abc"</code> , <code>":55"</code>
<code>Tank</code> , <code>Module</code> , <code>CrewMember</code>	Classes. <code>Module.oxygenTank</code> and <code>CrewMember.module</code> are optional
<code>lab</code> , <code>hab</code> , <code>dock</code>	Modules. Tanks: Lab = 40, Hab = 12, Dock has no tank
<code>crew</code> , <code>roster</code>	4 crew members as an array and as <code>[String: CrewMember]</code>
Solution section	Commented-out signatures for every task, in order

The crew:

Name	Role	Priority	Module
Timur	Engineer	3	Lab
Dana	Scientist	4	Dock
Aigerim	Commander	1	Hab
Nurlan	Pilot	2	none (open space)

Level 1 · Decoding Telemetry

Task 1.1 — `parseReading(_)` (1 point)

Write the function:

```
func parseReading(_ raw: String) -> Reading?
```

A line is valid **only if all of these hold**:

- it contains `:`
- the sensor name (left side) is not empty
- the value (right side) is an integer
- the value is `>= 0`, **except** for the `TEMP` sensor (temperature can be negative)

Required: use a **single** `guard let` with multiple comma-separated conditions.

```
parseReading("02:87")    // (sensor: "02", value: 87)
parseReading("TEMP:-12") // (sensor: "TEMP", value: -12)
parseReading("RAD:-1")   // nil
parseReading(":55")       // nil
```

Task 1.2 — `parseLog(_)` (0.5 points)

```
func parseLog(_ lines: [String]) -> (valid: [Reading], invalidCount: Int)
```

Returns a tuple: all valid readings and the number of corrupted lines.

Code fragment A = `invalidCount` for `rawLog`.

Level 2 · Analysis

Task 2.1 — Your own filter (1 point)

We can't use `filter`. So we'll write our own:

```
func select(_ readings: [Reading], where isIncluded: (Reading) -> Bool) -> [Reading]
```

And a helper:

```
func values(of readings: [Reading]) -> [Int]
```

Get all readings from the `02` sensor. The `select` call must use **a trailing closure and `$0`**.

Task 2.2 — `stats` (1 point)

The `minMax` function from the lecture slides crashes on an empty array. Ours won't.

```
func stats(of values: [Int]) -> (min: Int, max: Int, average: Double)?
func stats(_ values: Int...) -> (min: Int, max: Int, average: Double)?
```

- Returns `nil` for empty input.
- The second (variadic) function **must not duplicate logic** — it calls the first one. Its body is a single line without `return`.

```
stats(3, 8, 1) // (min: 1, max: 8, average: 4.0)
stats()        // nil
```

Code fragment B = the average `02` value, converted to `Int`.

Task 2.3 — The Closure Ladder (0.5 points)

Sort the valid readings by `value` in **descending** order in five ways, each shorter than the last:

1. Full closure syntax with types and `return`
2. Types inferred from context
3. Implicit `return`
4. Shorthand argument names `$0`, `$1`
5. Trailing closure

All five results must match — verify this in code, not by eye.

Level 3 · Temperature Stabilization

Safe range: **18 to 24 inclusive**.

Task 3.1 — Protocols as values (0.5 points)

Write three functions of type `(Int) -> Int`:

- `heatUp` — +5 degrees
- `coolDown` — -3 degrees
- `hold` — no change

And a function that **returns a function**:

```
func chooseProtocol(for temp: Int) -> (Int) -> Int
```

Below 18 → `heatUp`, above 24 → `coolDown`, otherwise → `hold`.

Task 3.2 — `runUntilStable` (1 point)

```
func runUntilStable(from start: Int, maxSteps: Int = 10) -> (finalTemp: Int, steps: Int, isStable: Bool)
```

While the temperature is out of range and steps are fewer than `maxSteps`: pick a protocol with `chooseProtocol`, apply it, increment the counter.

```
runUntilStable(from: 31) // (finalTemp: 22, steps: 3, isStable: true)
runUntilStable(from: -100, maxSteps: 5) // (finalTemp: -75, steps: 5, isStable: false)
```

Code fragment C = `steps` for the **lowest** valid temperature in the log (find it using your `select` + `stats`, don't type it in by hand).

Level 4 · The Crew

Task 4.1 — `oxygenLevel(of:)` (0.5 points)

```
func oxygenLevel(of member: CrewMember) -> Int?
```

The function body is **one line**, using optional chaining.

Task 4.2 — `status(of:)` (1 point)

```
func status(of member: CrewMember) -> String
```

Situation	Result
Level < 20	"Aigerim: 12% CRITICAL"
Level ≥ 20	"Timur: 40% OK"
No data, but has a module	"Dana: no data (Dock)"
No module	"Nurlan: no data (open space)"

Required: use `guard let` and the `??` operator. Print the status of the whole crew.

Task 4.3 — Oxygen Transfer (1 point)

```
@discardableResult
func transferOxygen(from source: inout Int, to target: inout Int, amount: Int) -> Int
```

- A tank holds a maximum of 100.
- You can't take more than the source has, or add more than the target can hold.
- A negative `amount` → nothing happens.
- Returns how much was actually transferred.

Transfer **30** units from `Lab` to `Hab`. Remember: `oxygenTank` is optional, and `!` is forbidden.

Code fragment D = the oxygen level in `Hab` after the transfer.

Task 4.4 — Evacuation Order (0.5 points)

```
func evacuationOrder(_ names: String..., roster: [String: CrewMember]) -> [String]
```

Returns the names of found crew members, sorted by `priority`. Unknown names are skipped with a console message (`guard let + continue`).

```
evacuationOrder("Dana", "Ghost", "Aigerim", "Timur", roster: roster)
// Unknown crew member: Ghost
// ["Aigerim", "Timur", "Dana"]
```

Level 5 · The Saboteur's Logbook (0.5 points)

Someone on the crew wrote this code. It compiles, but it will doom the station. It is already in your starter file, commented out.

```
func reportOxygen(for member: CrewMember) -> String {
    let tank = member.module!.oxygenTank!
    return "\(member.name): \(tank.level)%"
}

func firstCritical(in crew: [CrewMember]) -> String {
    var result: String?
    for member in crew {
        if oxygenLevel(of: member)! < 20 {
            result = member.name
        }
    }
    return result!
}
```

1. Find **all** the problems (there are more than it seems — not every bug is a `!`).
2. For each one, write a comment: what data breaks the code, and what happens.
3. Rewrite both functions without `!`. `firstCritical` must return `String?`.
4. Write a test that **proves** the logic bug is fixed (hint: it isn't visible with the starter data).

Finale: The Launch Code

```
let launchCode = "(A)-(B)-(C)-(D)"
print("LAUNCH CODE: \(launchCode)")
```

Every value must be **computed** by your functions. Code where fragments are typed in as numbers won't be accepted.

Defense Questions (1 point)

Answer as comments at the end of your playground. Keep it short, in your own words, with code examples where needed.

1. How does `guard let` differ from `if let` **beyond syntax**? Give an example where `if let` would make the code noticeably worse.
2. Why can't you call `stats(someArray)` where `someArray: [Int]`, even though inside the function `values` is already `[Int]`?
3. Why doesn't `transferOxygen(from: &x, to: &x, amount: 5)` compile? What bug does this prevent?
4. Why doesn't `oxygenLevel(of: dana) ?? "no data"` compile?
5. What is the **full type** of the function `chooseProtocol` itself? Write it out and explain how to read it.

Bonus (+1 point, explore on your own)

Closures can **capture** variables from their surrounding scope. We didn't cover this in the lecture.

```
func makeAlarm(threshold: Int) -> (Int) -> Bool
```

Returns a closure that takes an oxygen level, returns `true` if the level is below the threshold, **and counts how many times the alarm has fired**, printing `"Alarm #N"`.

```
let alarm = makeAlarm(threshold: 20)
alarm(12) // Alarm #1 → true
alarm(40) // false
alarm(5)  // Alarm #2 → true
```

Explain: where does the counter physically live if `makeAlarm` has already returned?

Grading

Part	Points
Level 1 — Telemetry	1.5
Level 2 — Analysis	2.5
Level 3 — Temperature	1.5
Level 4 — Crew	3
Level 5 — Saboteur	0.5
Defense questions	1
Total	10
Bonus	+1
Each ! outside Level 5	−0.5
Wrong launch code	−1

Good luck. The crew is counting on you.